P1498R0
Constrained Internal Linkage for Modules

Published Proposal, 2019-02-20

This version:
http://wg21.link/p1498

Issue Tracking:
Inline In Spec

Authors:
Chandler Carruth
Nathan Sidwell
Richard Smith

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose constraints on the use of internal linkage names within module interface units to make them both reliable and useful. This is intended to avoid the need of linkage promotion and its associated problems while preserving the simple, teachable utility of internal linkage such as being able to factor non-inline function definitions into internal helpers without any non-local effects.

Table of Contents

§ 1. Problem & Motivation

Internal linkage names for functions, variables, and types are routinely used to factor and express implementation details of C++ interfaces today, and this practice is useful and important for maintainability of this code. One consequence of introducing modules into C++ is that it easily allows embedding the implementation of an interface into a module's interface unit without these being exposed to the consumers of that interface unit. In effect, these implementation details remain local to the translation unit, despite this being a module interface unit. However, as currently specified, using internal linkage names for components of these implementation details creates problems because those names may also be used in ways that do *not* remain local to the translation unit.

§ 2. Proposal

The overarching principle is that internal linkage names may only be used within a context that is defined to remain local to the translation unit for importable translation units.

§ 2.1. Translation Unit Local

Some parts of a module interface unit do not escape that translation unit. We call these *translation unit local* or *TU-local*. The canonical example is a non-inline function definition. The contents of this definition have *no* visible effect beyond the translation unit. Our current understanding of the TU-local constructs in C++:

- Types, functions, and variables whose names have internal linkage (including those within anonymous namespaces)
- Types, functions, and variables whose names have no linkage

  > ISSUE 1 Local class member functions appear to have no linkage even when they occur within inline functions with external (or module) linkage. This seems like a bug generally, but certainly such member functions are **not** TU-local.

- Function definitions which are not inline, non-templated, and not `consteval`
- Variable definitions which are not inline and non-templated
- Template specializations for which any template argument involves a TU-local entity
- Template (or nested with a template) type definitions, function definitions, and variable initializations for which implicit instantiation is precluded, perhaps through an explicit instantiations

> Note: an explicit instantiation of a class template might not preclude implicit instantiation of some of its member function definitions!

TODO(all): Are there missing items from this list?

2.2. Restrict Usage of Internal Linkage Names

We propose restricting the usage of an internal linkage name to TU-local contexts as defined above. This is not influenced by reachability making it both easy to check and easy to teach. It also matches the underlying use case of internal linkage names: organizing the implementation details of a translation unit. If they are implementation details, the should remain local.

We specifically mean to restrict *any* use except for the specific exception to ODR-use afforded for non-volatile const objects in [basic.def.odr]p12.2.1.

> Note: This stronger than ODR-use restriction is necessary to avoid forcing the creation of unique names even in contexts where no symbol reference is required. One example is referencing an internal linkage name from the signature of a function.

This restriction is only enforced for importable translation units which include module interface units, partition units, and header units. One specific advantage of this solution to the problems discovered with linkage promotion is that this rule can be consistently used across all importable units including header units. Non-importable units are discussed below.

2.2.1. Examples with basic functions

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
10 static int f_internal() { return f(); } // OK
11        int f_module()   { return f(); } // OK
12 export int f_exported() { return f(); } // OK
13
14 static inline int f_internal_inline() { return f(); } // OK
15        inline int f_module_inline()   { return f(); } // ERROR
16 export inline int f_exported_inline() { return f(); } // ERROR
17
18 static constexpr int f_internal_constexpr() { return f(); } // OK
19        constexpr int f_module_constexpr()   { return f(); } // ERROR
20 export constexpr int f_exported_constexpr() { return f(); } // ERROR
21
22 static consteval int f_internal_consteval() { return f(); } // OK
23        consteval int f_module_consteval()   { return f(); } // ERROR
24 export consteval int f_exported_consteval() { return f(); } // ERROR
25
26        decltype(f()) f_module_decltype()   { return 0; } // ERROR
27 export decltype(f()) f_exported_decltype() { return 0; } // ERROR
```

§ 2.2.2. Examples with basic templates

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
30 template <typename T> static int ft_internal() { return f(); } // OK
31 template <typename T>        int ft_module()   { return f(); } // ERROR
32 template <typename T> export int ft_exported() { return f(); } // ERROR
33 template <typename T>        int ftei_module() { return f(); } // OK for
34 template                     int ftei_module<int>();
35 template <typename T> export int ftei_exported() { return f(); } // OK f
36 template                     int ftei_exported<int>();
37
38 template <typename T> inline int ftei_module_inline() { return f(); } //
39 template                     int ftei_module_inline<int>();
40
41 template <typename T>
42 constexpr int ftei_module_constexpr() { return f(); } // ERROR
43 template int ftei_module_constexpr<int>();
```

§ 2.2.3. Examples with class member functions

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
46 namespace {
47 struct c_internal {
48   int mf();
49   int mf_internal_inline() { return f(); } // OK
50 };
51 int c_internal::mf() { return f(); } // OK
52 } // namespace
53
54 struct c_module {
55   int mf_module();
56   int mf_module_inline() { return f(); } // ERROR
57 };
58 int c_module::mf_module() { return f(); } // OK
59
60 export struct c_exported {
61   int mf_exported();
62   int mf_exported_inline() { return f(); } // ERROR
63 };
64 int c_exported::mf_exported() { return f(); } // OK
```

§ 2.2.4. Examples with class template member functions

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
67 namespace {
68 template <typename T> struct ct_internal {
69   int ct_mf();
70   int ct_mf_internal_inline() { return f(); } // OK
71 };
72 template <typename T>
73 int ct_internal<T>::ct_mf() { return f(); } // OK
74 }
75
76 template <typename T> struct ct_module {
77   int ct_mf_module();
78   int ct_mf_module_inline() { return f(); } // ERROR
79 };
80 template <typename T>
81 int ct_module<T>::ct_mf_module() { return f(); } // ERROR
82
83 export template <typename T> struct ct_exported {
84   int ct_mf_exported();
85   int ct_mf_exported_inline() { return f(); } // ERROR
86 };
87 template <typename T>
88 int ct_exported<T>::ct_mf_exported() { return f(); } // ERROR
89
90 export template <typename T> struct ctei_exported {
91   int ctei_mf_exported();
92   int ctei_mf_exported_inline() { return f(); } // ERROR
93 };
94 template <typename T>
95 int ctei_exported<T>::ctei_mf_exported() { return f(); } // OK for int
96 template struct ctei_exported<int>;
```

§ 2.2.5. Examples with variables

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
99  static int v_internal = f(); // OK
100        int v_module   = f(); // OK
101 export int v_exported = f(); // OK
102
103 static inline int v_internal_inline = f(); // OK
104        inline int v_module_inline   = f(); // ERROR
105 export inline int v_exported_inline = f(); // ERROR
106
107 struct c_sdm_module {
108   static int sdm_module;
109   static constexpr int sdm_module_constexpr = f(); // ERROR
110 };
111 int c_sdm_module::sdm_module = f(); // OK
```

Note: variable templates follow identical patterns as function templates.

§ 2.2.6. Examples with lambdas

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
115 // Note that this function is not inline, but the lambda's call operato
116 // inline and that type becomes exported as the return type.
117 export auto f_exported_lambda() { return [] { return f(); }; } // ERROR
```

§ 2.2.7. Examples with function local classes

```
1 export module M;
2
3 static constexpr int f() { return 0; }
```

...

```
120 static int flc_internal() {
121   struct lc_internal {
122     int lc_mf_internal() { return f(); } // OK
123   };
124   return lc_internal().lc_mf_internal();
125 }
126
127 int flc_module() {
128   struct lc_module {
129     int lc_mf_module() { return f(); } // OK
130   };
131   return lc_module().lc_mf_module();
132 }
133
134 export int flc_exported() {
135   struct lc_exported {
136     int lc_mf_exported() { return f(); } // OK
137   };
138   return lc_exported().lc_mf_exported();
139 }
140
141 static inline int flc_internal_inline() {
142   struct lc_internal_inline {
143     int lc_mf_internal_inline() { return f(); } // OK
144   };
145   return lc_internal_inline().lc_mf_internal_inline();
146 }
147
148 inline int flc_module_inline() {
149   struct lc_module_inline {
150     int lc_mf_module_inline() { return f(); } // ERROR
151   };
152   return lc_module_inline().lc_mf_module_inline();
153 }
154
155 export inline int flc_exported_inline() {
156   struct lc_exported_inline {
157     int lc_mf_exported_inline() { return f(); } // ERROR
158   };
159   return lc_exported_inline().lc_mf_exported_inline();
160 }
```

§ 2.2.8. Examples where template instantiation has interesting implications

```
export module M;

static constexpr int f() { return 0; }
```

...

```
// An anonymous type with an internal name.
namespace { struct t_internal {}; }

export template <typename T> int f_exported_instantiated() {
  return f(); // ERROR
}

// This instantiaties the exported template locally, but the important
// that to allow this instantiation to succeed, even though it is local
// require either instatiating it with internal linkage, which makes th
// contingent upon the contents of the function template definition. If
// to use the template definition linkage, we must compute a unique nam
// `t_internal` which would require linkage promotion.
static int sink = f_exported_instantiated<t_internal>();

// If it is possible to detect the linkage of an entity, then this temp
// breaks one of the previous options by choosing whether the instantia
// definition contains a usage of an internal name by observing the lin
// the declaration. This suggests computing the linkage based on the
// instantiated definition is a bad strategy.
export template <typename T> int f_exported_circular_linkage() {
  if constexpr (/* sfinae or reflection test for external linkage */) {
    return f();
  }
  return 0;
}
```

§ 2.3. Enforcing and Diagnosing the Restriction

We believe this restriction can be checked and enforced immediately for all non-templated contexts. For templated contexts, the restriction can be checked at the end of the translation unit in the vast

majority of cases. In the case of a templated context where there are explicit instantiations with the TU (or potentially some other obscure corner cases), the error will need to be deferred to instantiation. However, in those cases *all* instantiations outside of the TU will require this diagnostic, allowing for extremely simple implementation strategies. Implementations could simply emit deleted definitions rather than the problematic ones, although a higher quality implementation would be likely (including information to produce a good diagnostics).

§ 2.4. Addressing [p1395r0] Issues With Linkage Promotion and Partitions

The problems raised in [p1395r0] fundamentally arise from attempting to do linkage promotion and organizing the code of a module within partitions. Names with internal linkage or no linkage may *require* that property, and promoting them conflicts with this. With this proposal, we preclude linkage promotion, which precludes the issues in this paper.

§ 2.5. Addressing [p1347r1] Issues With Transitively Reachable Internal Names

The problems raised in [p1347r1] are described in terms of ADL-enabled reaching of internal linkage names from exported interfaces or exported inline function definitions. While the paper's examples and presentation focused on a particular mechanism of reaching this problematic behavior that appears to be precluded by the current wording, that only precludes the discussed mechanism for arriving at the problems -- the fundamental problem remains.

Unfortunately, this means the analysis of the problem space is less complete than we would like. It requires constructing much more complex examples to explore the space, and we are still trying to complete this exploration. However, so far every example we have thought of is addressed. At worst, we expect to potentially still need a smaller and more narrow fix for any aspects of this issue that remain even in the absence of linkage promotion. All of the primary paths we have examined are addressed by this change.

§ 2.6. Non-importable Translation Units

We believe these rules are desirable even in non-importable translation units. Within those contexts, ODR-uses of internal linkage names is a common source of ODR violations in the wild. As a consequence, the rule we suggest for importable translation units can and should be consistently taught to programmers for C++ as a whole.

We propose to deprecate all of the usages that are restricted above for importable units within non-importable units to put users on notice that C++ is moving away from supporting these patterns as part of the move towards modules.

2.7. Inline

We also suggest refining the *non-normative* meaning of the term *inline* for functions and variables within interface units. Currently, this is primarily associated with a hint to the optimizer to inline more aggressively. Increasingly, these hints are insufficient for peak performance and are replaced with profile guided inlining or stronger and non-semantic vendor specific hints. The hinting use case remains important, but we believe it would be better served by a separate construct that does not carry additional semantic impact and can be better tailored to the purpose of this hint.

Within interface units of modules, we suggest converging on an interperation more firmly rooted in the semantics: an inline entity (function definition or variable initialization) is semantically incorporated into (or *inlined* into) the interface of a module. The inlined code's behavior is now *part of* the interface and not an implementation detail. Changing its behavior *changes the interface*. The module interface now includes the specific behavior of this inlined code. Using this part of the interface may enable optimizations such as inlining as well as other optimizations.

We do not think this is a meaningfully different interpretation from the reality of inline as it is used and implemented today. It does enable optimization techniques (but not the fundamental optimizations). However, it shifts to a *semantic* basis.

We see this as a first (small) step on a longer path to decouple the semantic decision of inlining an implementation into an interface from a non-semantic decision of hinting to the optimizer about the utility and importance of a particular (as-if) transformation. The remaining path toward fully arriving at this more principled end state is outlined as future work.

2.8. Future Work

2.8.1. Introduce a non-semantic inlining hint annotation

Currently, the `inline` specifier is used in contexts that shouldn't be restricted in their usage of internal names because it also provides a potential hint to optimizers. This hint is often weaker than desired due to its pervasive (semantic) usage. Vendors have experimented with an explicitly non-semantic hint with the ability to also be a stronger hint. We should standardize such a hint, likely as an attribute.

This should also address the other problem with the `inline` specifier being relied on for hinting to the optimizer -- often times that hint is needed on the *call* rather than the *declaration*, making a specifier completely inapplicable.

2.8.2. Deprecate inline definitions of internal names

The direction of this change also suggests deprecating the usage of `inline` when declaring names with internal or no linkage. We are happy to provide a proposal to this effect if there is interest in EWG, but it should be done much more slowly and cautiously. While this is a bug-prone pattern due to the potential for ODR violations, it remains in use and we would need to carefully evaluate the impact on existing code.

2.8.3. Deprecate declaring new inline entities within implementation units

The direction of this change also suggests deprecating the usage of `inline` when declaring new names within an implementation unit. We are happy to provide a proposal to this effect if there is interest in EWG, but it should be done much more slowly and cautiously. This is not an especially bug-prone pattern, but merely surprising and out of step with the semantic model. As a consequence, we again would only suggest this with an appropriately long time horizon and careful communication to users to understand and minimize negative impact.

3. Alternatives

3.1. Require Names in Importable Units Have Non-Internal Linkage

Rather than restricting the usage of names with internal or no linkage within importable translation units, we could simply disallow names with internal or no linkage to be declared at all within these units.

However, definitions that are TU-local are an important facility introduced by modules, and users expect to be able to leverage internal functions to factor and manage code for such definitions. We should not create barriers to moving code into modules if we can avoid it and this proposal does not seem significantly more costly to achieve.

§ 3.2. Alternative Proposed Solutions in [p1395r0]

There are two alternatives suggested in [p1395r0]:

> 1. Unrestricted linkage promotion at the expense of restricting refactoring of module partition source that contains such promotion.
> 2. Unrestricted module partition refactoring at the expense of linkage promotion collisions.

Both of these are phrased as trade-offs and have significant disadvantages described in the paper.

§ 3.3. Alternative Proposed Solutions in [p1347r1]

One proposed solution is to simple make the internal names not visible outside of the translation unit. This has significant downsides due to causing the same code to be accepted both inside and outside of that translation unit but with different overloads selected. This at least seems prone to ODR violations as well as being deeply surprising.

Another proposed solution is to make internal names visible, but when selected by overload resolution the result be ill-formed. This avoids the risk of surprising differences at the cost of increased complexity.

§ 4. Wording

TODO(herring, sidwell): Provide wording.

§ 5. Acknowledgements

These ideas were refined with help from members of EWG and other discussions including at least Mathias Stearn, Davis Herring, Michael Spencer, and Daveed Vandevoorde. For anyone I have missed, apologies, and don’t hesitate to suggest an addition.

§ 6. Revision History

§ 6.1. Revision 0

Initially published during Kona 2019.

§ References

§ Informative References

[P1347R1]

Nathan Sidwell, Davis Herring. Modules: ADL & Internal Linkage. 17 January 2019. URL: https://wg21.link/p1347r1

[P1395R0]

Nathan Sidwell. Modules: Partitions Are Not a Panacea. 18 January 2019. URL: https://wg21.link/p1395r0

§ Issues Index

ISSUE 1 Local class member functions appear to have no linkage even when they occur within inline functions with external (or module) linkage. This seems like a bug generally, but certainly such member functions are **not** TU-local. ↵